Τεχνικές βελτιστοποίησης

Project
Ελαχιστοποίηση συνάρτησης πολλών μεταβλητών
Γενετικοί αλγόριθμοι

Λαβδάκης Γεώργιος
ΑΕΜ:10672

Contents

1 **Εισαγωγή** 3

2 **Θέμα 1: Μαθηματική διατύπωση προβλήματος** 5

3 **Θέμα 2: Υλοποίηση γενετικού αλγορίθμου** 6

4 **Θέμα 3: Υλοποίηση δεύτερου γενετικού αλγορίθμου** 10

1 Εισαγωγή

Το project αφορά την μελέτη γενετικών αλγορίθμων. Οι γενετικοί αλγοριθμοι στοχεύουν στην εύρεση βέλτιστων λύσεων σε πολύπλοκα προβλήματα τα οποία έχουν πολλές παραμέτρους και άρα δεν μπορούν να εφαρμοστούν εύκολα οι κλασσικές μέθοδοι βελτιστοποίησης. Αποτελούν τεχνικές ολικής βελτιστοποίησης και είναι εμπνευσμένοι από τη βιολογία. Συγκεκριμένα, χρησιμοποιούν την ιδέα της εξέλιξης μέσω γενετικής μετάλλαξης, φυσικής επιλογής και διασταύρωσης, επιλέγοντας τα άτομα με την καλύτερη απόδοση ώστε να μεταβιβάσουν τα χαρακτηριστικά τους στους απογόνους. Έτσι, με το πέρασμα των γενεών ο πληθυσμός των ατόμων εξελίσσεται προς τα άτομα με την καλύτερη απόδοση και η διαδικασία αυτή επαναλαμβάνεται μέχρι να εντοπιστεί η βέλτιστη λύση.

Οι γενετικοί αλγόριθμοι ξεκινάνε με ένα αρχικό πλήθος από τυχαίες λύσεις του προβλήματος, που ικανοποιούν τις προδιαγραφές του και ονομάζεται πληθυσμός (population). Κάθε άτομο του πληθυσμού αντιπροσωπεύει ένα διάνυσμα λύσης του προβλήματος το οποίο ονομάζεται χρωμόσωμα (chromosome). Τα χρωμοσώματα αυτά, όπως και στη φύση, εξελίσσονται μέσω επαναλήψεων που ονομάζονται γενεές (generations) και κάθε γενιά αξιολογείται με βάση μια συνάρτηση ικανότητας (fitness function). Για τη δημιουργία κάθε επόμενης γενιάς, των απογόνων (offspring), χρησιμοποιούνται οι πράξεις της διασταύρωσης (crossover), μετάλλαξης (mutation) και επιλογής (selection) με βάση τη fitness function. Μετά από αρκετές γενεές, ο αλγόριθμος εντοπίζει το καλύτερο χρωμόσωμα, το οποίο αποτελεί την βέλτιστη ή υποβέλτιστη λύση στο πρόβλημα, σύμφωνα με τη συνθήκη τερματισμού $|\frac{f(x_{k-1})-f(x_k)}{f(x_{k-1})}| < \epsilon$ με ϵ την επιθυμητή ακρίβεια της λύσης. Παρακάτω φαίνεται η δομή ενός γενετικού αλγορίθμου.

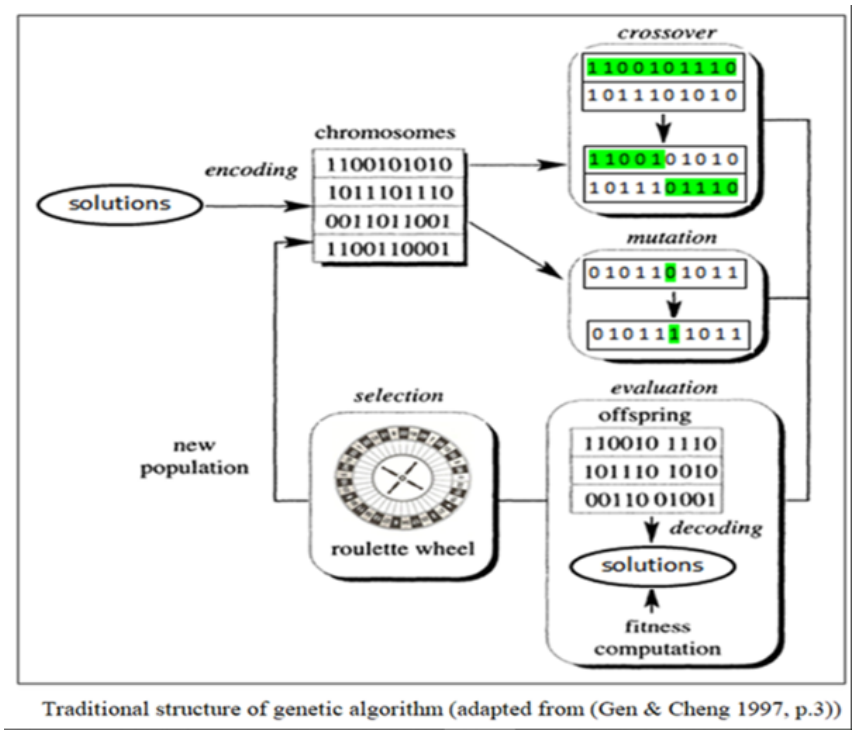


Figure 1.1: Δομή ενός γενετικού αλγορίθμου

2 Θέμα 1: Μαθηματική διατύπωση προβλήματος

Στόχος είναι η ελαχιστοποίηση του χρόνου διάσχισης του δικτύου ως προς τον ρυθμό διέλευσης οχημάτων στον δρόμο $i \in [1, 17]$. Επομένως, επιλέγεται συνάρτηση προς ελαχιστοποίηση η

$$f(x) = \sum_{i=1}^{17} x_i \cdot T_i$$

όπου x_i [οχ./min] ο ρυθμός διέλευσης οχημάτων στο δρόμο i και T_i ο χρόνος κίνησης στον ίδιο δρόμο. Ο T_i συναρτήσει του x_i είναι

$$T_i(x_i) = t_i + a_i \frac{x_i}{1 - \frac{x_i}{c_i}}[min]$$

όπου a_i σταθερές, c_i [οχ./min] ο μέγιστος δυνατός ρυθμός διέλευσης οχημάτων από τον δρόμο i, t_i [min] ο σταθερός χρόνος που απαιτείται για να κινηθούμε στον ίδιο δρόμο όταν η κίνηση είναι ασθενής και $V = 100$ [οχ./min] ο ρυθμός εισερχόμενων οχημάτων στο δίκτυο. Ο t_i είναι σταθερός άρα, η τιμή του δεν επηρεάζει την ελαχιστοποίηση της συνάρτησης. Επίσης, επειδή οι γενετικοί αλγόριθμοι είναι αλγόριθμοι μεγιστοποίησης τελικά ως fitness function ορίζεται η $\frac{1}{f(x)}$.

Για για να αποφευχθεί η συγκέντρωση οχημάτων στους κόμβους του δικτύου είναι επιθυμητό όσα οχήματα εισέρχονται σε κάθε κόμβο τόσα και να εξέρχονται. Επιπλέον, ο x_i δεν πρέπει να ξεπερνάει τον c_i γιατί τότε ο χρόνος διάσχισης της διαδρομής γίνεται άπειρος. Σύμφωνα με αυτές τις δύο συνθήκες και με βάση τη μορφή του δικτύου, προκύπτουν οι παρακάτω περιορισμοί

$\cdot x_1 + x_2 + x_3 + x_4 = V = 100$
$\cdot x_1 - x_5 - x_6 = 0$
$\cdot x_2 - x_7 - x_8 = 0$
$\cdot x_3 + x_8 + x_9 - x_{11} - x_{12} - x_{13} = 0$
$\cdot x_4 - x_9 - x_{10} = 0$
$\cdot x_6 + x_7 + x_{13} - x_{14} - x_{15} = 0$
$\cdot x_{10} + x_{11} - x_{17} = 0$
$\cdot x_5 + x_{14} - x_{16} = 0$
$\cdot x_{12} + x_{15} + x_{16} + x_{17} = V = 100$

3 Θέμα 2: Υλοποίηση γενετικού αλγορίθμου

Παρακάτω παρουσιάζεται ένας γενετικός αλγόριθμος στο Matlab που να λύνει το πρόβλημα. Επιλέγεται κωδικοποίηση με πραγματικές τιμές (real coded genetic algorithm) και κάθε χρωμόσωμα να είναι ένα διάνυσμα 1x17 που περιέχει όλα τα $x_i, i \in [1, 17]$. Το αρχείο στο οποίο υλοποιείται ο αλγόριθμος είναι το **GeneticAlgorithm.m**.

· Αρχικά, δημιουργείται η πρώτη γενιά με τη συνάρτηση **firstGeneration.m**. Κάθε επόμενη γενιά δημιουργείται με τη συνάρτηση **nextGeneration.m**.

· Ο έλεγχος των περιορισμών γίνεται με τη συνάρτηση **constraints.m**, η οποία επιστρέφει 0 αν κάποιος δεν ικανοποιούνται και 1 αν ικανοποιούνται όλοι. Για τον έλεγχο των ισοτικών περιορισμών π.χ.
$x_1 + x_2 + x_3 + x_4 = 100$, ελέγχονται οι ανισότητες π.χ.
$x_1 + x_2 + x_3 + x_4 - 100 < \epsilon$ όπου $\epsilon = 10^{-8}$ το μέγιστο ανεκτό σφάλμα.

· Μετά, η fitness function, στο αρχείο **fintessFunction.m**, αξιολογεί κάθε χρωμόσωμα του πληθυσμού επστρέφοντας μία βαθμολογία για το καθένα (fitness value) η οποία αντιπροσωπέυει την αποτελεσματικότητα της λύσης. Οσο μεγαλύτερη η τιμή που επιστρέφει η fitness function τόσο καλύτερο είναι και το αποτέλεσμα, δηλαδή τόσο πιο κοντά είναι η λύση στην βέλτιστη.

· Καθώς η διαδικασία επιλογής στους γενετικούς αλγορίθμους γίνεται στοχαστικά, κανένα χρωμόσωμα στον πληθυσμό, όσο ικανό και να είναι, δεν περναέι στην επόμενη γενιά με βεβαιότητα. Γι΄ αυτό, η διαδικασία επιλογής γίνεται με τον τροχό της ρουλέτας. Η επιλογή με ρουλέτα, στο αρχείο **rouletteSelection.m**, αρχικά υπολογίζει το συνολικό fitness του πληθυσμού, βρίσκει τη πιθανότητα επιβίωσης κάθε χρωμοσώματος, η οποία είναι ανάλογη του fitness value του $P_i = \frac{fitness\,value_i}{total\,fitness}$ και υπολογίζει την αθροιστική πιθανότητά του. Μετά, αφού ταξινομήσει τις πιθανότητες σε αύζουσα σειρά, παράγει τυχαίο αριθμό στο διάστημα $[0, 1]$ και επιλέγεται το χρωμόσωμα με αθροιστική πιθανότητα μεγαλύτερη ή ίση στον τυχαίο αριθμό. Έτσι, επιλέγεται η δεύτερη γενιά. Ουσιαστικά μοιράζεται ο τροχός της ρουλέτας σε τμήματα και αν ένα χρωμόσωμα έχει μεγάλο fitness value, καταλαμβάνει μεγαλύτερο μέρος του κύκλου και άρα έχει περισσότερες πιθανότητες να επιλεχθεί ενώ αν ένα χρωμόσωμα έχει μικρό fitness value καταλαμβάνει μικρότερο τμήμα και άρα είναι πιο δύσκολο να επιβιώσει.

· Η διαδικασία της διασταύρωσης υλοποιείται στη συνάρτηση **crossover.m**. Με τη διασταύρωση συνδυάζονται δύο τυχαία χρωμοσώματα, δύο γονείς και παράγονται απόγονοι (offspring). Οι κύριοι μέθοδοι διασταύρωσης είναι δύο. Στην πρώτη χρησιμοποιείται η μέση τιμή των χρωμοσωμάτων των δύο γονέων, δημιουργώντας με τον τρόπο αυτόν δύο ίδιους απογόνους. Η μέθοδος αυτή αναφέρεται ως μέθοδος της μέσης τιμής. Στη δεύτερη μέθοδο παράγονται δύο απόγονοι, καθένας από τους οποίους έχει κληρονομήσει ένα τμήμα από τον ένα γονέα και το υπόλοιπο από τον άλλο. Η διαδικασία αυτή αναφέρεται ως διακριτή διασταύρωση. Η δεύτερη μέθοδος χρησιμοποιείται συνήθως όταν η κωδικοποίηση του χρωμοσώματος έχει γίνει δυαδικά άρα, αφού έχει επιλεχθεί κωδικοποίηση με πραγματικές τιμές χρησιμοποιείται η πρώτη μέθοδος. Η συνάρτηση δηλαδή, έχει ως είσοδο δύο τυχαία χρωμοσώματα (parent1, parent2) και δημιουργεί δύο νέα ίδια, στο οποίο καθένα στοιχείο $x_i, i \in [1, 17]$ είναι ίσο με $\frac{x_{1i}+x_{2i}}{2}$. Στο τέλος, ελέγχεται αν ικανοποιούνται οι περιορισμοί του προβλήματος με τη συνάρτηση constraints και αν δεν ικανοποιούνται, η αλλαγή απορρίπτεται.

· Η διαδικασία της μετάλλαξης υλοποιείται στη συνάρτηση **mutation.m** με τη βοήθεια των συναρτήσεων **distributeChange.m** και **adaptToChange.m**. Με τη μετάλλαξη κάποιο τυχαίο χρωμόσωμα μετασχηματίζεται ώστε να υπάρχει ποικιλία στον πληθυσμό. Συγκεκριμένα, για κάθε χρωμόσωμα ξεχωριστά και για τυχαία επιλεγμένα πεδία του χρωμοσώματος αυτού, μεταβάλλεται η ποσότητα x_i κατά ένα Δx_i, το οποίο ακολουθεί Gaussian κατανομή, σύμφωνα με τη σχέση $x_i(k) = x_i(k-1) + \Delta x_i(k)$. Λόγω των αρχικών περιορισμών του προβλήματος, μεταλλάξεις μπορούν να γίνουν μόνο στα x_1, x_2, x_3, x_4 αφού, κάθε μεταβολή που συμβαίνει σε $x_i, i \in [5, 17]$ είναι αποτέλεσμα μιας μεταβολής που έγινε στους αρχικούς κλάδους $i = 1, 2, 3, 4$. Μια μεταβολή δηλαδή π.χ. στο x_4 προκαλεί αλλαγή στα x_1, x_2, x_3 για να εξακολουθεί να ισχύει ο περιορισμός $x_1 + x_2 + x_3 + x_4 = 100$ και στα x_9, x_{10} για να εξακολουθεί να ισχύει ο περιορισμός $x_4 - x_9 - x_{10} = 0$. Η μεταβολή αυτή τελικά, προκαλεί αλλαγές σε όλους τους κλάδους του δικτύου. Η μετάλλαξη ενός κλάδου βέβαια, δεν έχει ως αποτέλεσμα τη μεταβολή όλων των κλάδων που αναφέρονται στον περιορισμό απαραίτητα, αρκεί ένας τουλάχιστον κλάδος να αλλάξει ώστε να εξακολουθεί να ισχύει ο περιορισμός. Για το παραπάνω παράδειγμα δηλαδή, η μεταβολή $x_4 + \Delta x_4$ μπορεί να προκαλέσει μία από τις παρακάτω αλλαγές ώστε να εξακολουθεί να ισχύει $x_1 + x_2 + x_3 + x_4 = 100$
1) $x_1 - \Delta x_4$

2) $x_2 - \Delta x_4$
3) $x_3 - \Delta x_4$
4) $x_1 - k \cdot \Delta x_4$ και $x_2 - (1-k) \cdot \Delta x_4$ μοιράζοντας έτσι τη μεταβολή σε δύο κλάδους (k τυχαίος αριθμός στο διάστημα [0, 1]),
5) $x_1 - k \cdot \Delta x_4$ και $x_3 - (1-k) \cdot \Delta x_4$
6) $x_2 - k \cdot \Delta x_4$ και $x_3 - (1-k) \cdot \Delta x_4$
7) $x_1 - \frac{k_1}{k_1+k_2+k_3} \cdot \Delta x_4$ και $x_2 - \frac{k_2}{k_1+k_2+k_3} \cdot \Delta x_4$ και $x_3 - \frac{k_3}{k_1+k_2+k_3} \cdot \Delta x_4$ μοιράζοντας έτσι τη μεταβολή σε τρεις κλάδους (k_i τυχαίοι αριθμοί στο διάστημα [0, 1]). Αντίστοιχα προκύπτουν οι περιπτώσεις για τα x_1, x_2, x_3.
Στη συνάρτηση mutation επιλέγεται τυχαία ποιό από τα x_1, x_2, x_3, x_4 θα μεταλλαχθεί και με τη συνάρτηση distributeChange μοιράζεται τυχαία η μεταβολή σε τουλάχιστον ένα από τα υπόλοιπα. Όταν η μεταβολή μοιράζεται σε 2 ή παραπάνω κλάδους το ποσοστό της μεταβολής που θα πάει σε κάθε κλάδο επιλέγεται και πάλι τυχαία. Μετά την μετάλλαξη, καλείται η adaptToChange η οποία μοιράζει εκ νέου τη μεταβολή στα υπόλοιπα $x_i, i \in [5, 17]$, έτσι ώστε να ικανοποιούνται όλοι οι περιορισμοί. Στο τέλος, ελέγχεται αν ικανοποιούνται οι περιορισμοί του προβλήματος με τη συνάρτηση constraints και αν δεν ικανοποιούνται, η αλλαγή απορρίπτεται.

· Παρακάτω παρουσιάζεται το γράφημα του fitness value συναρτήσει των γενεών και εικόνα από το command line που περιέχει τον αριθμό επαναλήψεων που χρειάστηκαν για να τερματίσει ο αλγόριθμος, το τελικό χρωμόσωμα, τον ελάχιστο χρόνο διάσχυσης του δικτύου και αν η λύση είναι έγκυρη ή όχι.

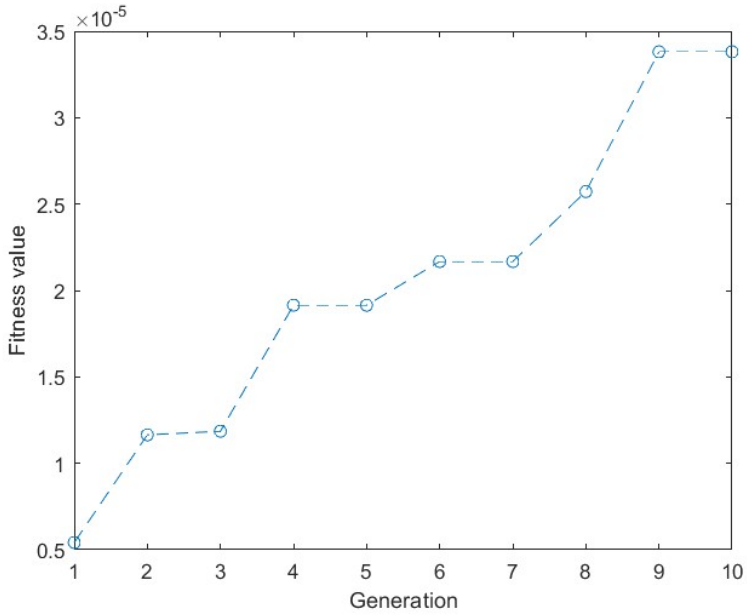


Figure 3.1: Το fitness value συναρτήσει των γενεών για τον πρώτο αλγόριθμο

```
The number of iterations is 14
The final chromosome is
  Columns 1 through 12

   36.0695   11.2207   21.1288   31.5810   23.6406   12.4289    5.5224    5.6983   11.9583   19.6226    7.9752    8.4706

  Columns 13 through 17

   22.3398    8.8799   31.4111   32.5205   27.5978

Total time is 886.9367
Solution is valid.
```

Figure 3.2: Αποτελέσματα αλγορίθμου

Τα δέκα χρωμοσώματα της πρώτης γενιάς παράγονται τυχαία άρα, τα αποτελέσματα του αλγορίθμου είναι διαφορετικά σε κάθε εκτέλεση. Τα παραπάνω αποτελούν ενδεικτικά παραδείγματα. Παρατηρείται ότι με το πέρασμα των γενεών το fitness value του πληθυσμού αυξάνεται δηλαδή, ο αλγόριθμος κατευθύνεται στη βέλτιστη λύση. Επίσης, μετά από αρκετές επαναλήψεις προκύπτει ότι ο ελάχιστος χρόνος είναι κοντά στα 900 sec. Κάποιες φορές όμως, ο αλγόριθμος φαίνεται πως εγκλωβίζεται σε τοπικό ελάχιστο και προκύπτουν μεγαλύτερες τιμές, έως 1300 sec.

4 Θέμα 3: Υλοποίηση δεύτερου γενετικού αλγορίθμου

Στο θέμα 3, αλλάζει το V και παύει πια να θεωρείται σταθερό. Πλέον η τιμή του V που αντιπροσωπεύει τον ρυθμό εισερχόμενων οχημάτων στο δίκτυο είναι μεταβλητή και μπορεί μάλιστα να μεταβάλλεται μέχρι ± 15. Άρα, το κάθε χρωμόσωμα πλέον αποτελείται από 18 στοιχεία δηλαδή είναι ένα διάνυσμα 1x18 που περιέχει όλα τα $x_i, i \in [1, 17]$ και το V ως 18η παράμετρο. Το αρχείο στο οποίο υλοποιείται ο αλγόριθμος είναι το **GeneticAlgorithm2.m**.

Οι αλλαγές που χρειάστηκε να γίνουν είναι λίγες. Αρχικά, για την δημιουργία της πρώτης γενιάς, γίνεται τυχαία επιλογή για το V και έπειτα γίνεται το μοίρασμα σε όλες τις κατευθύνσεις όπως πριν (συνάρτηση **firstGeneration2.m**). Επίσης, στην συνάρτηση της μετάλλαξης πλέον μπορεί να αλλάξει και η 18η παράμετρος και άρα να επιλεχθεί V βέλτιστο, ενώ κατά τα άλλα ακολουθεί την λογική που αναφέρθηκε προηγουμένως (συνάρτηση **mutation2.m**). Στη διασταύρωση δεν χρειάστηκαν ιδιαίτερες αλλαγές, μόνο αύξηση του μεγέθους του χρωμοσώματος.

· Παρακάτω παρουσιάζεται το γράφημα του fitness value συναρτήσει των γενεών και εικόνα από το command line που περιέχει τον αριθμό επαναλήψεων που χρειάστηκαν για να τερματίσει ο αλγόριθμος, το τελικό χρωμόσωμα, τον ελάχιστο χρόνο διάσχυσης του δικτύου, το βέλτιστο V (η τελευταία τιμή του χρωμοσώματος) αν η λύση είναι έγκυρη ή όχι.

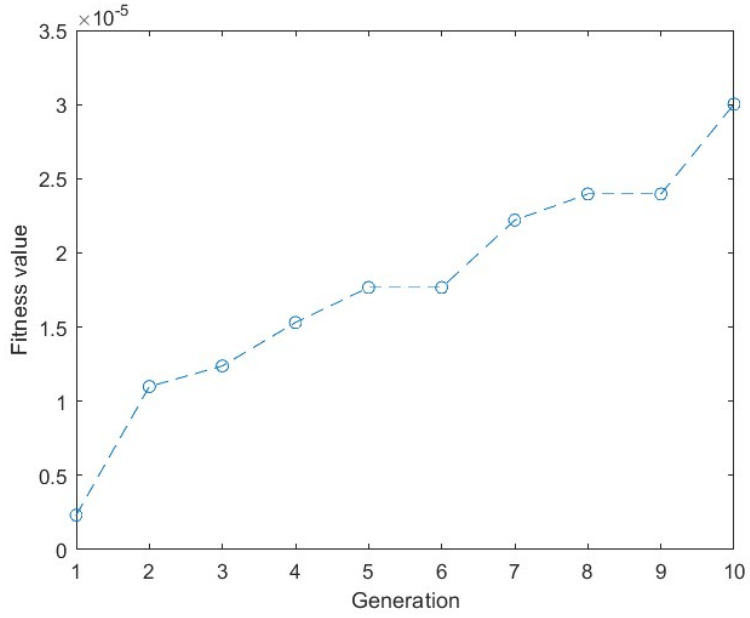


Figure 4.1: Το fitness value συναρτήσει των γενεών για τον δεύτερο αλγόριθμο

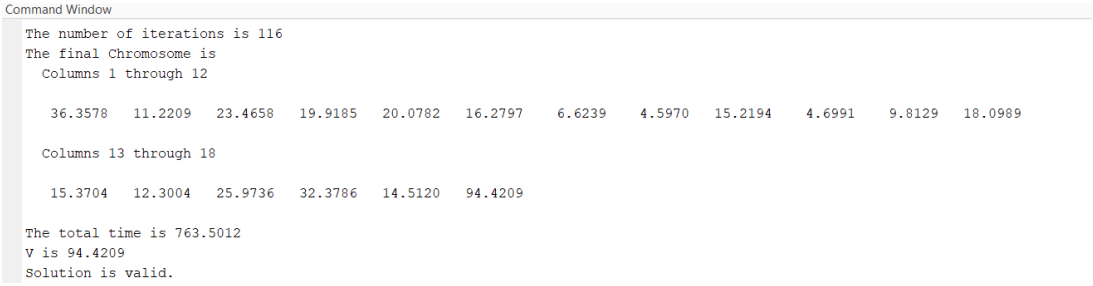


Figure 4.2: Αποτελέσματα αλγορίθμου

Όπως στον πρώτο αλγόριθμο, τα δέκα χρωμοσώματα της πρώτης γενιάς παράγονται τυχαία άρα, τα αποτελέσματα του αλγορίθμου είναι διαφορετικά σε κάθε εκτέλεση. Τα παραπάνω αποτελούν ενδεικτικά παραδείγματα. Η τελική τιμή του V, σχεδόν πάντα καταλήγει να είναι μικρότερη του 100, το οποίο είναι και το αναμενόμενο αποτέλεσμα. Αυτό έχει σαν αποτέλεσμα να υπάρχει λιγότερη κίνηση στο δίκτυο και τελικά, να προκύπτει μικρότερη τιμή συνολικού χρόνου διάσχυσής του σε σχέση με τον πρώτο αλγόριθμο. Ωστόσο, πιθανόν λόγω της αύξησης του μεγέθους του χρωμοσώματος, ο αλγόριθμος εγκλωβίζεται πιο εύκολα σε τοπικά ελάχιστα και προκύπτουν μεγάλες τιμές ελάχιστου χρόνου. Ακόμα, σε σχέση με τον πρώτο αλγόριθμο κάποιες φορές οι λύσεις που προκύπτουν δεν ικανοποιούν τους αρχικούς περιορισμούς του προβλήματος και δεν είναι έγκυρες (στο τέλος του αλγορίθμου ελέγχεται μία τελευταία φορά αν ικανοποιούνται οι περιορισμοί του προβλήματος με τη συνάρτηση constraints και τυπώνεται "Solution is invalid." αν δεν ικανοποιούνται).